

Logistic Regression Analysis: Pima Indians Diabetes

Data Analysis Report

2026-05-19

Contents

Load Libraries	1
1. Environment Setup & Data Loading	2
2. Data Cleaning & Imputation	2
3. Exploratory Data Analysis (EDA)	2
4. Data Splitting	3
5. Model Estimation & Diagnostics	4
Collinearity Testing	4
Odds Ratios:	5
6. Out-of-Sample Performance Evaluation	5
Confusion Matrix Diagnostics	5
Discrimination Mapping (ROC & AUC)	6

Load Libraries

```
library(mlbench)    # For PimaIndiansDiabetes2
library(tidyverse)  # For data transformation & plots
library(caret)      # For partitioning & confusion matrix
library(ROCR)       # For ROC & AUC calculation
library(mice)       # For handling missing values via imputation
library(car)        # For VIF calculations
```

1. Environment Setup & Data Loading

First, we load the required libraries for our analytical pipeline. We use `mlbench` for our source data, `mice` for handling missing features, `tidyverse` for data manipulation/visualization, and `caret`/`ROCR` for cross-validation diagnostics.

```
data("PimaIndiansDiabetes2")
glimpse(PimaIndiansDiabetes2)

#> Rows: 768
#> Columns: 9
#> $ pregnant <dbl> 6, 1, 8, 1, 0, 5, 3, 10, 2, 8, 4, 10, 10, 1, 5, 7, 0, 7, 1, 1~
#> $ glucose <dbl> 148, 85, 183, 89, 137, 116, 78, 115, 197, 125, 110, 168, 139, ~
#> $ pressure <dbl> 72, 66, 64, 66, 40, 74, 50, NA, 70, 96, 92, 74, 80, 60, 72, N~
#> $ triceps <dbl> 35, 29, NA, 23, 35, NA, 32, NA, 45, NA, NA, NA, NA, 23, 19, N~
#> $ insulin <dbl> NA, NA, NA, 94, 168, NA, 88, NA, 543, NA, NA, NA, NA, 846, 17~
#> $ mass <dbl> 33.6, 26.6, 23.3, 28.1, 43.1, 25.6, 31.0, 35.3, 30.5, NA, 37.~
#> $ pedigree <dbl> 0.627, 0.351, 0.672, 0.167, 2.288, 0.201, 0.248, 0.134, 0.158~
#> $ age <dbl> 50, 31, 32, 21, 33, 30, 26, 29, 53, 54, 30, 34, 57, 59, 51, 3~
#> $ diabetes <fct> pos, neg, pos, neg, pos, neg, pos, neg, pos, pos, neg, pos, n~
```

2. Data Cleaning & Imputation

```
colSums(is.na(PimaIndiansDiabetes2))

#> pregnant glucose pressure triceps insulin mass pedigree age
#>          0          5          35          227          374          11          0          0
#> diabetes
#>          0
```

To avoid discarding partial information, we execute Multiple Imputation by Chained Equations (MICE) using Predictive Mean Matching (PMM) to fill missing data points based on surrounding patient trends.

```
set.seed(123)
# Run imputation routing
imputed_data <- mice(PimaIndiansDiabetes2, m = 1, method = "pmm", printFlag = FALSE)

# Build out the completed dataset
clean_diabetes <- complete(imputed_data)

sum(is.na(clean_diabetes))

#> [1] 0
```

3. Exploratory Data Analysis (EDA)

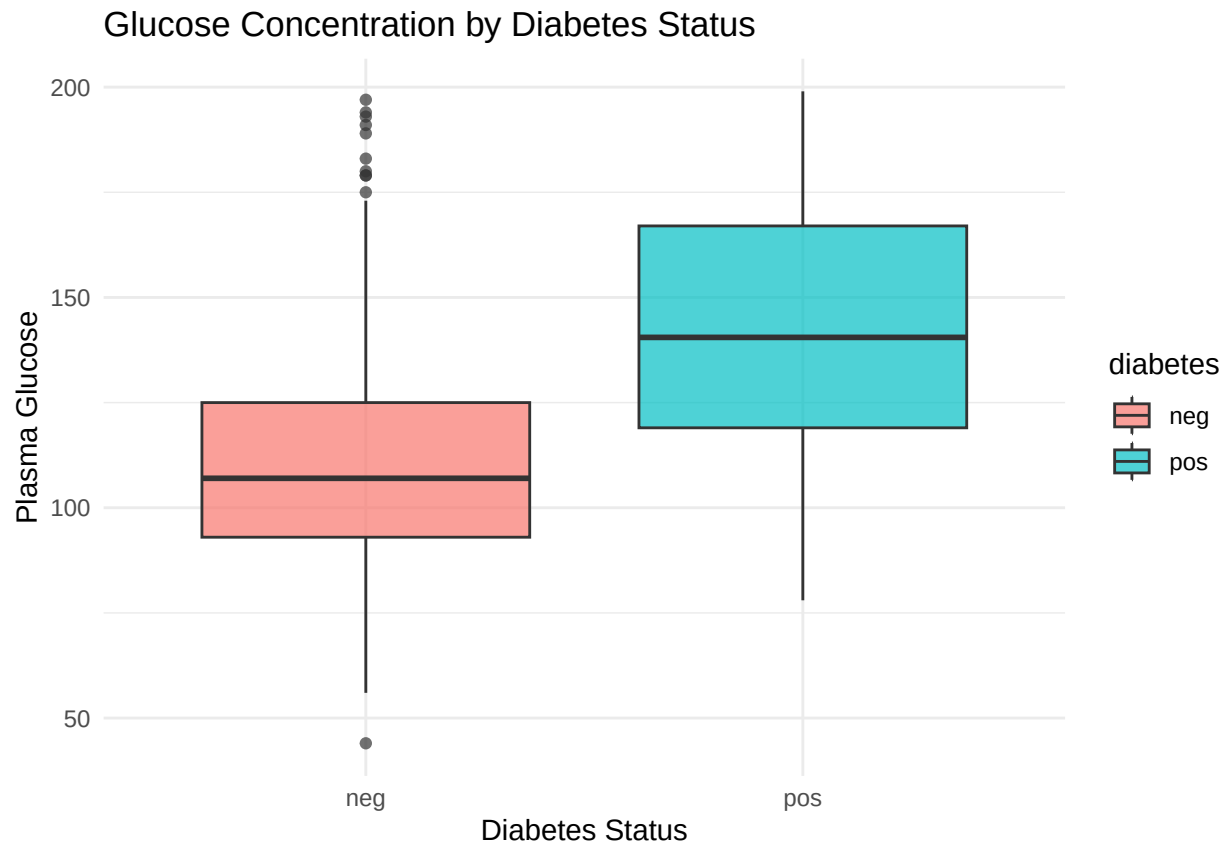
Before building our statistical equations, we check the baseline distribution of the target variable (diabetes) to establish class balances.

```
prop.table(table(clean_diabetes$diabetes))
```

```
#>  
#>      neg      pos  
#> 0.6510417 0.3489583
```

We now isolate plasma glucose values across positive and negative diagnostic states to visualize group variance.

```
ggplot(clean_diabetes, aes(x = diabetes, y = glucose, fill = diabetes)) +  
  geom_boxplot(alpha = 0.7) +  
  theme_minimal() +  
  labs(  
    title = "Glucose Concentration by Diabetes Status",  
    x = "Diabetes Status",  
    y = "Plasma Glucose"  
  )
```



4. Data Splitting

We perform an 80/20 train-test split using stratified sampling via caret to ensure equivalent base-rate metrics in both validation matrices.

```
set.seed(42)

# Generate partition indexes
train_index <- createDataPartition(clean_diabetes$diabetes, p = 0.8, list = FALSE)

train_set <- clean_diabetes[train_index, ]
test_set <- clean_diabetes[-train_index, ]
```

5. Model Estimation & Diagnostics

We construct a binary generalized linear model using the complete matrix of explanatory features under a logit link function.

```
diabetes_model <- glm(diabetes ~ ., data = train_set, family = binomial)
summary(diabetes_model)
```

```
#>
#> Call:
#> glm(formula = diabetes ~ ., family = binomial, data = train_set)
#>
#> Coefficients:
#>             Estimate Std. Error z value Pr(>|z|)
#> (Intercept) -9.4588942  0.9397847 -10.065  < 2e-16 ***
#> pregnant      0.1652091  0.0366958   4.502 6.73e-06 ***
#> glucose       0.0407534  0.0049334   8.261  < 2e-16 ***
#> pressure     -0.0045708  0.0095603  -0.478   0.633
#> triceps       0.0044097  0.0134558   0.328   0.743
#> insulin      -0.0008093  0.0011466  -0.706   0.480
#> mass          0.0871179  0.0219512   3.969 7.23e-05 ***
#> pedigree      1.0301626  0.3333315   3.091   0.002 **
#> age           0.0001159  0.0110795   0.010   0.992
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> (Dispersion parameter for binomial family taken to be 1)
#>
#>      Null deviance: 796.05  on 614  degrees of freedom
#> Residual deviance: 557.13  on 606  degrees of freedom
#> AIC: 575.13
#>
#> Number of Fisher Scoring iterations: 5
```

Collinearity Testing

We calculate the Variance Inflation Factor (VIF) to ensure that inter-variable correlations do not warp our standard error values.

```
vif(diabetes_model)
```

```
#> pregnant  glucose pressure  triceps  insulin    mass pedigree    age
#> 1.481160 1.464261 1.237956 1.625870 1.463801 1.816190 1.015777 1.629219
```

Note: No VIF metrics cross above the conservative benchmark threshold ($VIF > 5$). Structural multicollinearity is not an issue in this context.

Odds Ratios:

To evaluate effect sizing intuitively, we convert log-odds metrics to standard risk-based Odds Ratios (OR) alongside their 95% confidence bounds.

```
exp(cbind(OR = coef(diabetes_model), confint(diabetes_model)))
```

```
#>               OR      2.5 %      97.5 %
#> (Intercept) 7.799278e-05 1.151529e-05 0.0004612944
#> pregnant    1.179640e+00 1.098953e+00 1.2693018165
#> glucose     1.041595e+00 1.031896e+00 1.0520875367
#> pressure    9.954396e-01 9.769042e-01 1.0142894378
#> triceps     1.004419e+00 9.777745e-01 1.0309644392
#> insulin     9.991910e-01 9.969476e-01 1.0014600304
#> mass        1.091025e+00 1.045862e+00 1.1400777567
#> pedigree    2.801521e+00 1.467911e+00 5.4339688679
#> age         1.000116e+00 9.783402e-01 1.0218980808
```

6. Out-of-Sample Performance Evaluation

Now we apply our fitted parameters to the untouched validation test set. Predictions are generated as probabilities and converted to factor levels using the traditional 0.5 classification cutoff threshold.

```
# Calculate continuous risk probabilities
test_set$prob <- predict(diabetes_model, newdata = test_set, type = "response")

# Convert probabilities to factors
test_set$pred_class <- factor(ifelse(test_set$prob > 0.5, "pos", "neg"), levels = c("neg", "pos"))
```

Confusion Matrix Diagnostics

We calculate predictive indicators (Accuracy, Sensitivity, and Specificity) to evaluate spatial performance.

```
confusionMatrix(test_set$pred_class, test_set$diabetes, positive = "pos")
```

```
#> Confusion Matrix and Statistics
#>
#>              Reference
#> Prediction neg pos
#>      neg   87  23
#>      pos   13  30
#>
#>              Accuracy : 0.7647
#>              95% CI   : (0.6894, 0.8294)
#>      No Information Rate : 0.6536
#>      P-Value [Acc > NIR] : 0.001988
```

```

#>
#>           Kappa : 0.4563
#>
#> McNemar's Test P-Value : 0.133614
#>
#>           Sensitivity : 0.5660
#>           Specificity : 0.8700
#>           Pos Pred Value : 0.6977
#>           Neg Pred Value : 0.7909
#>           Prevalence : 0.3464
#>           Detection Rate : 0.1961
#>           Detection Prevalence : 0.2810
#>           Balanced Accuracy : 0.7180
#>
#>           'Positive' Class : pos
#>

```

Discrimination Mapping (ROC & AUC)

Finally, we run a threshold sweep across the true positive/false positive spectrum to establish discrimination power via the Area Under the Curve (AUC).

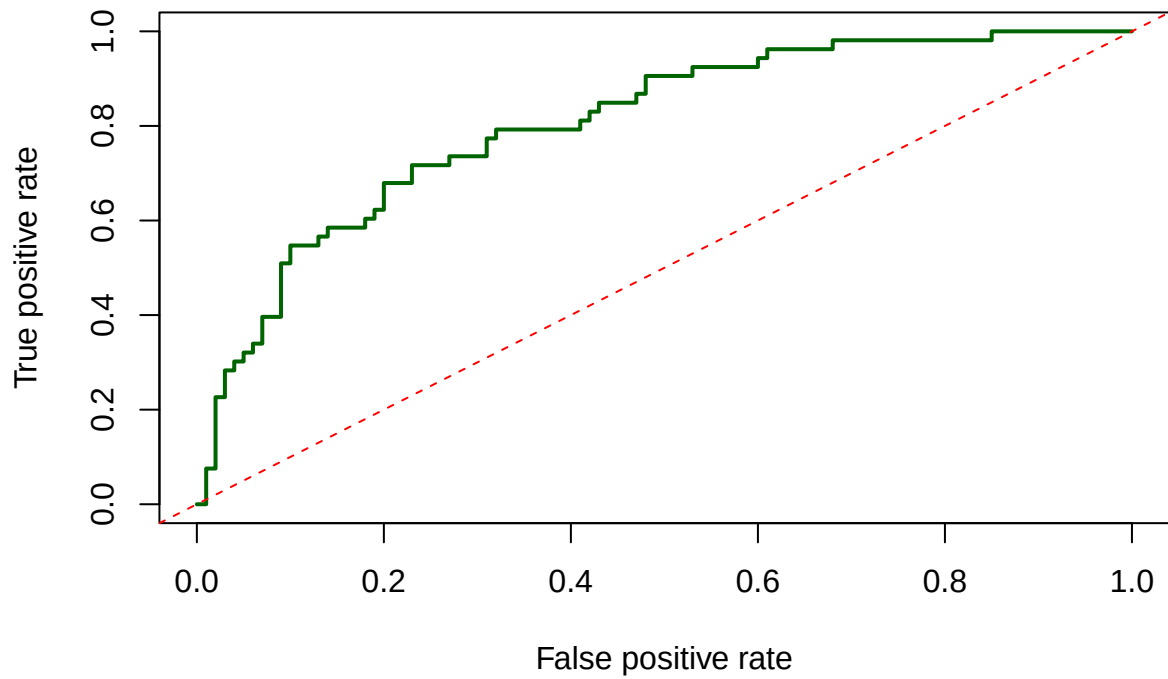
```

# Generate performance tracking profiles
pred_obj <- prediction(test_set$prob, test_set$diabetes)
perf_obj <- performance(pred_obj, measure = "tpr", x.measure = "fpr")

# Plot ROC Space
plot(perf_obj, col = "darkgreen", lwd = 2, main = "ROC Curve: Pima Indians Diabetes Model")
abline(a = 0, b = 1, lty = 2, col = "red") # Random baseline

```

ROC Curve: Pima Indians Diabetes Model



```
# Isolate numeric AUC value  
auc_obj <- performance(pred_obj, measure = "auc")  
cat("Final Area Under the Curve (AUC):", round(auc_obj@y.values[[1]], 4), "\n")
```

```
#> Final Area Under the Curve (AUC): 0.8064
```